Annexe N°1 - JSX

Introduction

Les concepteurs de **React** ont donc cherché un moyen d'alléger l'écriture et leur choix s'est porté sur l'utilisation de **JSX** (JavaScript eXtension). JSX est une forme d'écriture des éléments **React**, plus simple à lire et à écrire que les instructions **React.createElement()**. Cette syntaxe est donc abondamment utilisée dans les programmes **React**.

Au fur et à mesure, vous verrez que le **JSX** est un langage très intuitif à utiliser. Voici les deux propriétés de base pour ce qui est des balises utilisées :

- Toute balise commençant par une minuscule (**div**, **span**, **label**, etc.) est réservée aux éléments HTML. Ces éléments sont déclarés par **React DOM**, et vous obtiendrez une erreur si vous utilisez un élément inexistant.
- Toute balise commençant par une majuscule (**Greetings**, **App**, etc.) doit être déclarée explicitement, ce doit donc être un élément du scope courant : fonction déjà déclarée, composant importé d'une bibliothèque ou d'un autre fichier…

Hello React avec JSX

JSX est donc une forme nouvelle d'écriture des éléments **React**. Par exemple, voici un paragraphe contenant Hello React dans son texte :

```
/* Ceci est un commentaire */
let p = <p>Hello React</p>; // Code JSX
```

Pour afficher ce paragraphe, on utilise l'outil **Babel** pour interpréter le code **JSX** et de le transformer en interne en code JavaScript compréhensible par le navigateur. D'autres outils existent, nous utiliserons celui-ci.

La page HTML utilisant Babel s'écrit de la façon suivante, en intégrant notre code JSX :

Exemple 1 :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <script crossorigin
src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

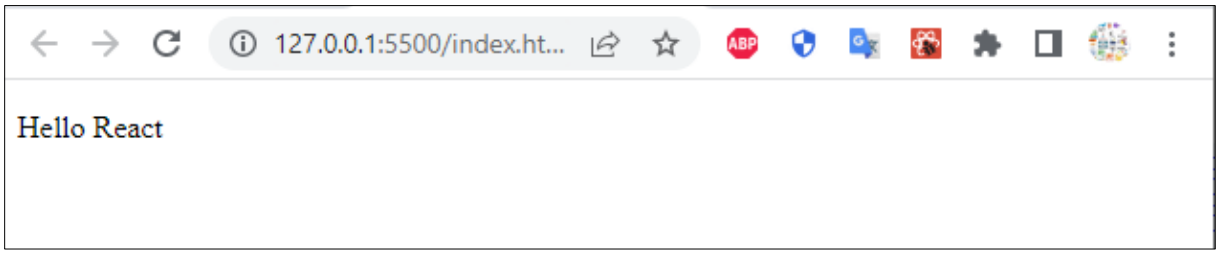
</head>

<body>

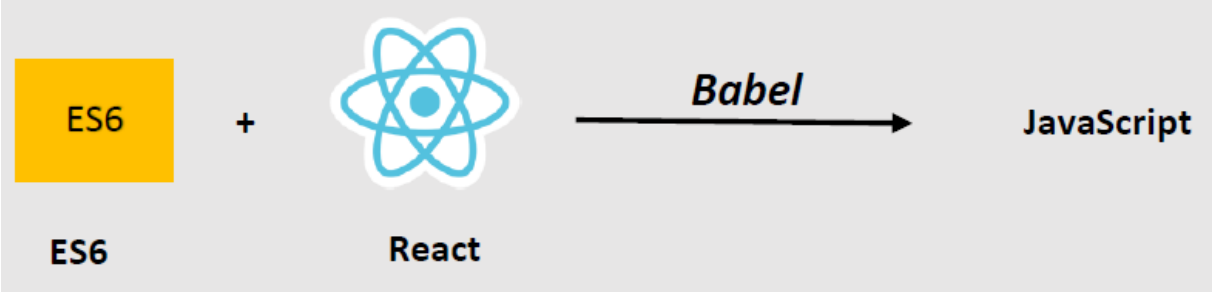
    <div id="app"></div>
```

```
    <script type="text/babel">
        let p = <p>Hello React</p>;
        ReactDOM.render(p, document.getElementById("app"));
    </script>

</body>
</html>
```



On inclut le fichier JavaScript de Babel au moyen de la balise **<script src="...">** (cela correspond à l'interpréteur qui traduira le code **JSX** en code JavaScript), puis on indique quelle partie du code JavaScript est à interpréter par **Babel**. Pour cela, on inclut l'attribut **type="text/babel"** dans la balise **<script>** contenant notre code JavaScript (et **JSX**) :



Le code JavaScript permettant la création des éléments **React** est écrit en **JSX** (et sera traduit en JavaScript pur par Babel), tandis que les éléments **React** ainsi créés seront insérés dans la page HTML au moyen de l'instruction **ReactDOM.render()**.

Remarque : l'inclusion de **Babel** pour interpréter le code **JSX** ralentit le programme, vu qu'une étape de traduction est nécessaire avant d'exécuter le code JavaScript. Par conséquent, l'utilisation de Babel ne peut être viable que dans le cadre de l'écriture du programme (en mode développement). Cet outil ne peut pas être utilisé dans le cadre d'un déploiement (mode production). Dans ce dernier cas, on utilisera d'autres outils tels que **Webpack** pour créer un package plus compact.

Créer une liste d'éléments en JSX

```
<script type="text/babel">
        const liste = <ul>
            <li>Apples</li>
            <li>Bananas</li>
            <li>Cherries</li>
        </ul >

        ReactDOM.render(liste, document.getElementById("app"));

</script>
```

On remarque qu'une même instruction peut s'écrire sur plusieurs lignes et doit obligatoirement commencer par une balise ouvrante et se terminer par balise fermante (ici, **
** et **
**).



Ajouts d'attributs dans le code JSX

Le code **JSX** comporte les éléments qui seront affichés dans la page HTML. Ces éléments peuvent avoir des attributs tels que **id**, **style** ou **className** (l'attribut **class** est remplacé par l'attribut **className**).

Commençons par ajouter les attributs **id** et **className**. Pour cela, on définit la classe **CSS red** dans la balise **<style>** de la page :

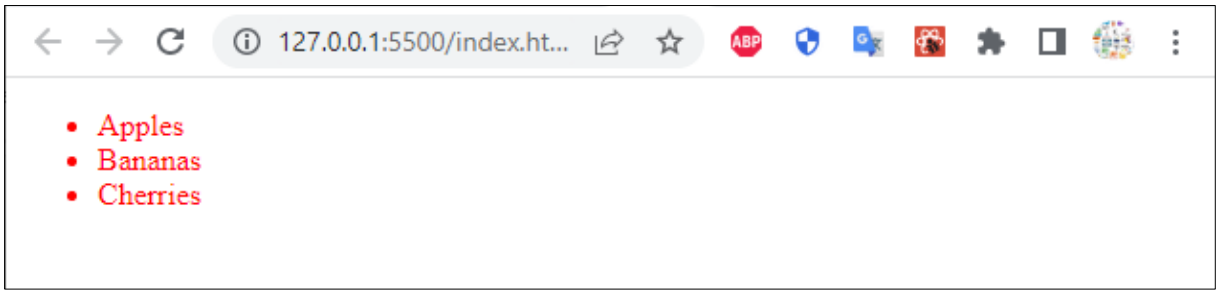
```
<style>
       .red {
           color : red;
       }
</style>
```

Maintenant on va définir les attributs **id** et **className** dans le code **JSX** :

```
<script type="text/babel">
        const liste = <ul id="liste1" className="red">
            <li>Apples</li>
            <li>Bananas</li>
            <li>Cherries</li>
        </ul >
```

```
        ReactDOM.render(liste, document.getElementById("app"));
</script>
```

La liste définie par <ul> possède bien l'**id** "**liste1**", tandis que la classe **CSS red** est bien définie sur la liste grâce à l'attribut **className** . Donc les éléments de liste sont de couleur rouge :



L'exemple précèdent sans utiliser le **JSX** sera écrits de la façon suivante :

```
<script type="text/babel">
       const liste = React.createElement("ul", { id: "liste1", className: "red" },
           React.createElement("li", null, "Apples"),
           React.createElement("li", null, "Bananas"),
           React.createElement("li", null, "Cherries"));
       ReactDOM.render(liste, document.getElementById("app"));
</script>
```

Ajout de l'attribut style en JSX

La syntaxe à utiliser pour insérer l'attribut style est légèrement différente. On souhaite maintenant définir dans le style de la liste (élément **<ul>**) la propriété CSS **list-style-type** et lui attribuer la valeur "**none**", ce qui signifie que les éléments de liste s'affichent sans être précédés par un point. La propriété **color** sera également définie dans le style à la valeur "**blue**" :

```
<script type="text/babel">
        const liste = <ul id="liste1" style={{listStyleType:"none",
color:"blue"}}>
            <li>Apples</li>
            <li>Bananas</li>
            <li>Cherries</li>
        </ul >

        ReactDOM.render(liste, document.getElementById("app"));

</script>
```

La propriété **list-style-type** s'écrit dans le code JavaScript sous la forme **listStyleType**, en remplaçant comme d'habitude chaque tiret et la lettre qui le suit par une majuscule (**listStyleType**). L'attribut **style** est défini en utilisant les caractères **{{** et **}}**, soit deux accolades ouvrantes puis deux fermantes. Les accolades extérieures indiquent que l'expression à l'intérieur est une expression JavaScript. De plus, le style doit dans ce cas être défini au moyen d'un objet JavaScript.



Utilisation d'instructions JavaScript dans le code JSX

On peut utiliser des instructions JavaScript dans du code **JSX**, à condition d'entourer les instructions JavaScript avec des accolades. Chaque instruction entourée d'accolades est évaluée par le navigateur, et son résultat est inséré en lieu et place de l'instruction JavaScript évaluée. Ceci permet de créer du code **JSX** qui s'adapte aux conditions définies dans le programme.

Voici un exemple qui permet de définir le style de la balise (**<ul>**), mais dans une instruction JavaScript :

```
<script type="text/babel">
        const mystyle = {listStyleType:"none", color :"red"};
        const liste = <ul id="liste1" style={ mystyle }>
            <li>Apples</li>
            <li>Bananas</li>
            <li>Cherries</li>
        </ul >

        ReactDOM.render(liste, document.getElementById("app"));

</script>
```

L'instruction JavaScript **{ mystyle }** indique de calculer la valeur de l'expression **mystyle**, puis d'affecter cette valeur au style de l'élément dans le code JSX.



Insérer les éléments d'un tableau dans une liste (<ul>)

Ici les éléments qui vont construire la liste (**<ul>**) sont placés dans un tableau **elements** qui est ensuite parcouru par le code JavaScript et JSX :

```
<script type="text/babel">
        var mycolor ="green";
        var mystyle ={listStyleType :"none", color:mycolor};
        const elements = ["Apples", "Bananas", "Cherries"];
        const liste = <ul id="liste1" style={mystyle}>
            {
                elements.map(elemnt => <li>{elemnt}</li>)
            }
            </ul>
        ReactDOM.render(liste, document.getElementById("app"));

</script>
```



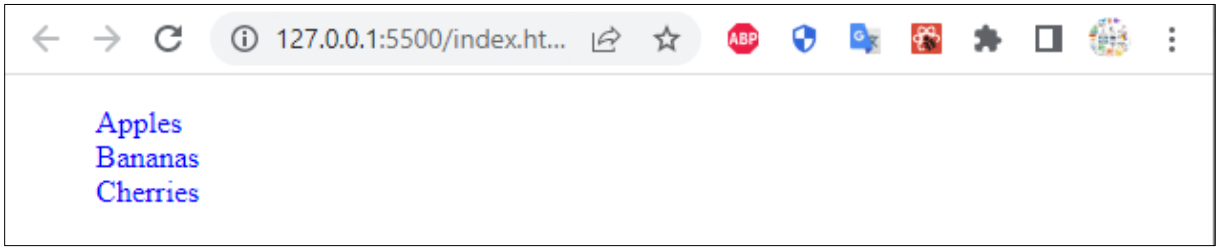
Créer une fonction qui retourne du code JSX

On va refaire l'exemple précédent à l'aide d'une fonction qui retourne au code JSX :

```
<script type="text/babel">
        var mycolor = "blue";
        var mystyle = { listStyleType: "none", color: mycolor };
        const elements = ["Apples", "Bananas", "Cherries"];
        const Liste = () => {
            return <ul id="liste1" style={mystyle}>
                {elements.map(element => <li>{element}</li>)}
            </ul>
        }

        ReactDOM.render(<Liste />, document.getElementById("app"));

</script>
```



Transmettre des attributs dans un élément JSX

Pour transmettre des attributs dans un éléments JSX, on utilise l'argument **props** passé dans un compostant **React**.

Voici un exemple :

```
<script type="text/babel">
        const Car = (progs) => {
            return (
                <div>
                    <h2>Brand title : {progs.brand} </h2>
                    <p>I am a : {progs.model} </p>
                </div>
            );
        }
        ReactDOM.render(<React.Fragment>
            <Car brand="Ford" model="Focus" />
            <Car brand="Renault" model="Clio" />
            <Car brand="Hyndai" model="Tucson" />
            </React.Fragment>, document.getElementById("app"));

</script>
```



L'exemple précédent peut s'écrire en déstructurant le paramètre **props** de la façon suivante :

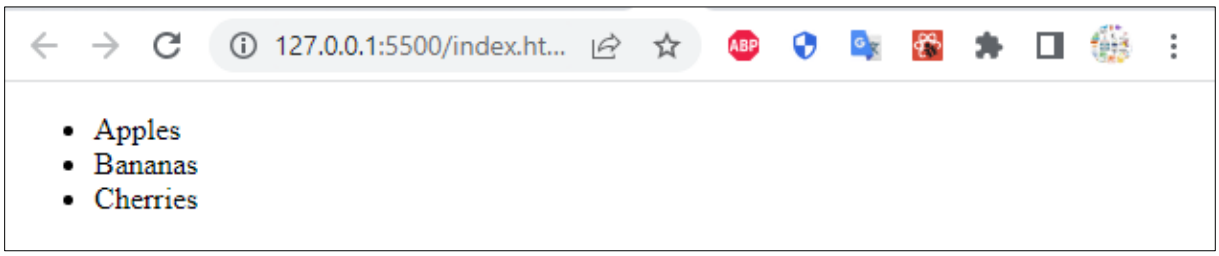
```
<script type="text/babel">
        const Car = ({brand, model}) => {
            return (
                <div>
                    <h2>Brand title : {brand} </h2>
                    <p>I am a : {model} </p>
                </div>
            );
        }
        ReactDOM.render(<React.Fragment>
            <Car brand="Ford" model="Focus" />
            <Car brand="Renault" model="Clio" />
            <Car brand="Hyndai" model="Tucson" />
            </React.Fragment>, document.getElementById("app"));

</script>
```

L'exemple suivant permet de créer une liste (**<ul>)** à partir des éléments d'un tableau :

```
<script type="text/babel">
   const elements = ["Apples", "Bananas", "Cherries"];
   const List = (props) => {
       return (
           <ul>
               {props.array.map(elm => <li>{elm}</li>)}
           </ul>
       );
   }
   ReactDOM.render(<List array={elements} />, document.getElementById("app"));

</script>
```



On peut aussi transmettre un sous élément à un élément JSX en utilisant **progs.children** :

```
<script type="text/babel">
        const Car = (progs) => {
            return (
                <div>
                    <h2>Brand title : {progs.brand} </h2>
                    <p>I am a : {progs.children} </p>
                </div>
            );
        }
        ReactDOM.render(<React.Fragment>
            <Car brand="Ford" >
                <span>Focus</span>
            </Car>
            <Car brand="Renault">
                <span>Clio</span>
            </Car>
            <Car brand="Hyndai">
                <span>Tucson</span>
            </Car>
        </React.Fragment>, document.getElementById("app"));

</script>
```



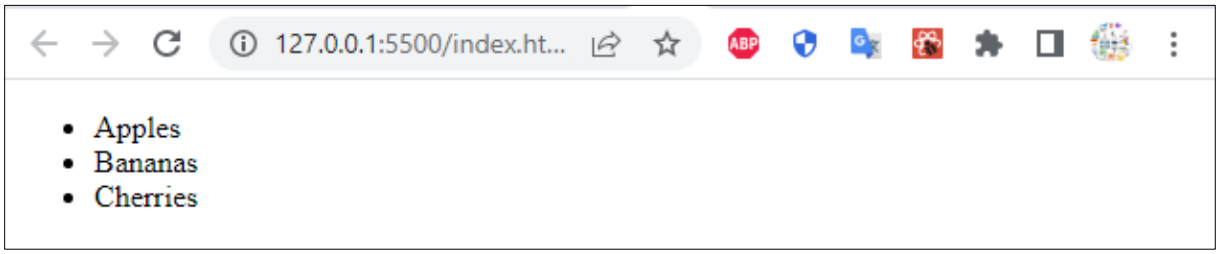
Créer un compostant JSX avec une classe :

On peut également créer des éléments **React** (et **JSX**) à partir d'une classe dérivant de la classe **React.Component**.

Voici un exemple :

```
<script type="text/babel">
        const elements = ["Apples", "Bananas", "Cherries"];
        class List extends React.Component {
            constructor(props) {
                super(props);
            }
            render() {
                return (
                    <ul>
                        {this.props.array.map(elm => <li>{elm}</li>)}
                    </ul>
                );
            }
        }
        ReactDOM.render(<List array={elements} />, document.getElementById("app"));

</script>
```



Remarque : la classe **List** hérite de la classe **React.Component**, dans ce cas le constructeur **constructor()** est accessible implicitement dans la classe **List** donc on peut tout simplement écrire la classe List de cette façon :

```
class List extends React.Component {
            render() {
                return (
                    <ul>
                        {this.props.array.map(elm => <li>{elm}</li>)}
                    </ul>
                );
            }
}
```

Utiliser des expressions conditionnelles dans le code JSX retourné

Il est possible d'utiliser des expressions conditionnelles avec ? et : dans le code JSX, à condition d'entourer l'ensemble avec des accolades **{** et **}**.

Voici un exemple :

```
<script type="text/babel">
        const List = (props) => {
            return (
                <ul>
                    {props.hidefirst ? null : <li>Fruits</li>}
                    <li>Apples</li>
                    <li>Bananas</li>
                    <li>Cherries</li>
                    <hr/>
                </ul >);
        }

        ReactDOM.render(<React.Fragment>
            <List hidefirst={false} />
            <List hidefirst={true} />
            </React.Fragment>, document.getElementById("app"));

    </script>
```

